

DSA Lab Exercise: Basic Doubly Linked List Operations

Title

Menu-Driven Implementation of a Doubly Linked List

Objective

To write a menu-driven C program to implement a **doubly linked list** and perform basic operations:

- Insert a node at the beginning
- Insert a node at the end
- Delete a node from the beginning
- Delete a node from the end
- Display the list with memory addresses

Problem Statement

Write a C program to create a doubly linked list using dynamic memory allocation. The program should provide a menu to perform insertion and deletion operations at both ends.

While displaying the list, show the **address of the node**, `prev` **address, data, and** `next` **address**.

For example:

Doubly Linked List

NODE ADDRESS	PREV ADDRESS	DATA	NEXT ADDRESS	

0x55a...	(nil)	10	0x55b...	
0x55b...	0x55a...	20	0x55c...	
0x55c...	0x55b...	30	(nil)	

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
```

```

    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Create a new node */
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->prev = NULL;
    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}

```

```

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    printf("%d inserted at beginning.\n", value);
}

```

```

/* Insert at end */

```

```

void insertEnd(int value)
{
    struct Node *newNode;
    struct Node *temp;

    newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
        newNode->prev = temp;
    }

    printf("%d inserted at end.\n", value);
}

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty!\n");
        return;
    }

    temp = head;
    head = head->next;

    if (head != NULL)
    {
        head->prev = NULL;
    }

    printf("%d deleted from beginning.\n", temp->data);

    free(temp);
}

```

```

/* Delete from end */
void deleteEnd()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty!\n");
        return;
    }

    temp = head;

    /* Move to the last node */
    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    if (temp->prev == NULL)
    {
        /* Only one node */
        head = NULL;
    }
    else
    {
        temp->prev->next = NULL;
    }

    printf("%d deleted from end.\n", temp->data);

    free(temp);
}

```

```

/* Display list with addresses */
void display()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty!\n");
        return;
    }

    temp = head;

    printf("\n");
    printf("Doubly Linked List\n");

```

```

printf("-----\n");

printf("| %-18s | %-18s | %-10s | %-18s |\n",
       "NODE ADDRESS",
       "PREV ADDRESS",
       "DATA",
       "NEXT ADDRESS");

printf("-----\n");

while (temp != NULL)
{
    printf("| %-18p | %-18p | %-10d | %-18p |\n",
          (void*)temp,
          (void*)temp->prev,
          temp->data,
          (void*)temp->next);

    temp = temp->next;
}

printf("-----\n");
}

/* Main function */
int main()
{
    int choice;
    int value;

    do
    {
        printf("\n=====\\n");
        printf("      DOUBLY LINKED LIST MENU\\n");
        printf("=====\\n");

        printf("1. Insert at Beginning\\n");
        printf("2. Insert at End\\n");
        printf("3. Delete from Beginning\\n");
        printf("4. Delete from End\\n");
        printf("5. Display\\n");
        printf("6. Exit\\n");

        printf("=====\\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

```

```

        printf("Enter value: ");
        scanf("%d", &value);

        insertBeginning(value);
        break;

    case 2:

        printf("Enter value: ");
        scanf("%d", &value);

        insertEnd(value);
        break;

    case 3:

        deleteBeginning();
        break;

    case 4:

        deleteEnd();
        break;

    case 5:

        display();
        break;

    case 6:

        printf("Program terminated.\n");
        break;

    default:

        printf("Invalid choice! Try again.\n");

    }

} while (choice != 6);

```

Explanation

Each node contains:

```

+-----+-----+-----+
| prev   | data   | next   |
+-----+-----+-----+

```

For example, after inserting 10 , 20 , and 30 :

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

The memory representation can be viewed as:

prev	data	next
↓	↓	↓
+-----+	+-----+	+-----+
NULL	0x....	0x....
10	20	30
0x....	0x....	NULL
+-----+	+-----+	+-----+

Important statements

Insert at beginning:

C

```

newNode->next = head;
head->prev = newNode;
head = newNode;

```

Insert at end:

C

```

temp->next = newNode;
newNode->prev = temp;

```

Delete from beginning:

C

```

head = head->next;
head->prev = NULL;

```

Delete from end:

C

```
temp->prev->next = NULL;
```

Display address:

C

```
printf("%p", (void*)temp);
```

The `%p` format specifier displays a **memory address**.

Sample Output

```
=====
          DOUBLY LINKED LIST MENU
=====
1. Insert at Beginning
2. Insert at End
3. Delete from Beginning
4. Delete from End
5. Display
6. Exit
=====
Enter your choice: 2

Enter value: 10
10 inserted at end.

Enter your choice: 2

Enter value: 20
20 inserted at end.

Enter your choice: 2

Enter value: 30
30 inserted at end.

Enter your choice: 5
```

Output:

Doubly Linked List

NODE ADDRESS	PREV ADDRESS	DATA	NEXT ADDRESS	

0x55a123...	(nil)	10	0x55a145...	
0x55a145...	0x55a123...	20	0x55a167...	

| 0x55a167... | 0x55a145... | 30 | (nil) |

Lab task: Students should execute the program, insert at least **5 nodes**, perform deletion from both ends, and observe how the `prev` and `next` addresses change after each operation.